

dsc_550_term_project

August 2, 2026

1 DSC 550 Term Project

1.1 Milestone 1

Problem:

The World Health Organization (WHO) published a statement that implored the use of the best available scientific or research evidence even over a physician's own opinion gained through their clinical experience to implement clinical decisions to improve patient outcomes^[1]. The Oxford Centre for Evidence-Based Medicine (CEBM) has stated that systematic reviews of randomized controlled trials are the best available evidence ^{[1],[2]}.

Despite the importance systematic reviews and meta-analyses are supposed to play in the field of medicine, there is a lack of consistency and quality in these types of studies available for physicians in the literature databases, exacerbated by the inefficiency by which they are completed, with the average time for completing and publishing systematic reviews from start to finish for research teams being 11.5 months and 16.2 months, respectively^[3]. Currently, most teams are still performing systematic reviews manually, with excel spreadsheets and printed full-text PDFs.

Fig 1

Table 1 Average Number of Authors, Time Spent, and Yield of Literature Search

Category	Mean	SD	Median	IQR	Min	Max
Number of authors	5	3	5	3	1	27
Average time spent	67.3	31	65.8	41.6	6	186
Yield ^a	2.94	6.49	0.93	2.5	0	64.71

a Yield = $\frac{\text{Records included}}{\text{Records found}}$

Table 2 Funded vs. unfunded projects

Category	Time	Authors
funded	42	6.8
not funded	26	4.8

Fig 2

On the other hand, research teams and software companies have developed automation tools to

shorten the amount of time it takes to complete these systematic review papers. In fact, one team has published an award-winning article that outlines how a systematic review paper was and can be completed in 2 weeks. However, the software and tools they used are largely inaccessible to the average researcher, produce inconsistent and inaccurate results. Furthermore, much of these automation tools rely on artificial intelligence, which do not have the intuition and sense that is required to bridge the evidence together with a practical solution to be applied. Research is filling in gaps of knowledge with new data, while artificial intelligence is designed to summarize what has already been filled with old data.

task	hours
Data extraction	461
Manuscript	428
Title abstract screening	404
Risk of bias	323
Citation screening	201
Full-text screening	187
Protocol	175
Meta-analysis	167
Trial registries screening	123
Search strategy	109
Forms	72
Search	72
Find previous or upcoming SR	63
PDFs	41
Deduplication	16

Therefore, I wanted to find a solution that meets these two somewhere in the middle. A **system** for conducting **systematic** reviews, where the user inputs dictate every nuance of automation, which does not rely on third party or artificial intelligence.

Fig 3

Conclusion (edit 02.08.2026)

Title and abstract screening takes researchers on average of 404 hours to complete. That is around 25 days per author, and 50.5 days for both authors to complete. In order to increase the efficiency of this step, while preserving the validity and accuracy of included and excluded studies, the term project's goal is to create a predictive model using a training dataset of manually screened records from PubMed/MEDLINE from a real-life, on-going research project. Then, using this predictive model, a systematic pipeline will be created to expedite the process that closes matches manually-performed results.

1.2 Milestone 2

Now that you have created your idea, located data, and have started your graphical analysis, you will move on to the data preparation process of your project. After completing Milestone 2, your data should be ready for the model building/evaluation phase.

Here is a list of steps to consider performing in Milestone 2:

- Drop any features that are not useful for your model building and explain why they are not useful.
- Perform any data extraction/selection steps.
- Transform features if necessary.
- Engineer new useful features.
- Deal with missing data (do not just drop rows or columns without justifying this).
- Create dummy variables if necessary.

Explain your process at each step. You can use any methods/tools you think are most appropriate. Do what makes the most sense for your data/problem. This will vary greatly among different projects. Be careful to avoid data snooping in these steps.

It is important to note that these milestones are meant to keep you on track for the final project submission. At any point, you can pivot or modify your project as needed based on what you discover. These milestones are not final versions; they are drafts of the many steps you need to complete along the way.

As a reminder, Teams is a great place to discuss your project with your peers. Feel free to solicit feedback/input (without creating a group project!) and collaborate on your projects with your peers.

Each milestone will build on top of each other, so make sure you do not fall behind. Submit Milestones 1 & 2 together. I recommend building your project milestones in a Jupyter Notebook, building upon one another. However, make sure it is clear where Milestone 1 ends and Milestone 2 begins.

```
[1]: import pandas as pd
import re
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import ENGLISH_STOP_WORDS, TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score,
    roc_curve,
    roc_auc_score,
    ConfusionMatrixDisplay,
    RocCurveDisplay
)
```

```
from sklearn.naive_bayes import MultinomialNB
```

1. Drop any features that are not useful for your model building and explain why they are not useful.

```
[2]: csv = ["excluded", "included", "irrelevant"]

for x in csv:
    df = pd.read_csv(f"./data/{x}.csv", encoding = "utf-8") # Import the
    ↪ exported PubMed/MEDLINE results from search.
    df = df.drop(columns = ["Published Month", "Issue", "Pages", "Accession",
    ↪ Number", "Ref", "Covidence #", "Notes", "Tags", "Volume", "Authors"])
    globals()[x] = df
```

The following features from PubMed/MEDLINE export were dropped: Published Month, Issue, Pages, Accession Number, Ref, Covidence #, Notes, Tags, and Volume. These features were not useful for the building the predictive model because since the goal is to use text classification for the Title and Abstract columns in order to find the words and phrases that best predict whether the study should be included or excluded from the review according to a predefined eligibility criteria for inclusion and exclusion of studies. The first step of the screening process involves Title and Abstract screening, and thus, only these two features were selected for building the model.

2. **Perform any data extraction/selection steps:** select features Title, Abstract, Published Year, and Journal, and code results Included, Excluded as 0, 1 as a Label column.

```
[3]: irrelevant["Label"] = 0 # Excluded during title and abstract screening
excluded["Label"] = 1 # Excluded during full-text screening
included["Label"] = 2 # Included into qualitative review

df = pd.DataFrame(columns = ["Study", "Title", "Abstract", "Published Year",
    ↪ "DOI", "Journal", "Label"])
df = pd.concat([df, included, excluded, irrelevant])
df.to_csv("./data/labeled_training_data.csv", encoding = "utf-8")
```

3. **Transform features if necessary:** create stemmed Title and Abstract columns using NLTK's PorterStemmer

```
[4]: from nltk.stem import PorterStemmer

PS = PorterStemmer()
stop_words = set(ENGLISH_STOP_WORDS)

def stemmer(column):
    words = re.sub(r"[^a-zA-Z]", " ", str(column).lower()).split()
    return " ".join(PS.stem(word) for word in words if word not in stop_words)

df["stemmed_title"] = df["Title"].apply(stemmer)
df["stemmed_abstract"] = df["Abstract"].apply(stemmer)
```

```
[5]: df.to_csv("./data/stemmed_training.csv", encoding = "utf-8")
```

4. Deal with missing data (do not just drop rows or columns without justifying this).

```
[23]: df = pd.read_csv("./data/stemmed_training.csv", encoding = "utf-8")
df["stemmed_title"] = df["stemmed_title"].fillna("")
df["stemmed_abstract"] = df["stemmed_abstract"].fillna("")
df["DOI"] = df["DOI"].fillna("")
```

5. Engineer new useful features: the stemmed title and stemmed abstract features were combined into one feature column.

```
[24]: df["stemmed_title_abstract"] = (df["stemmed_title"] + " " +
    ↪df["stemmed_abstract"])
```

6. Create dummy variables if necessary.

```
[21]: # df = pd.get_dummies(df, drop_first = True)
```

Create training and test datasets

```
[25]: df = df[df["Label"] < 2] # Include only title and abstract screening labels by
    ↪dropping rows that were excluded / included during full-text screening
df["Label"] = df["Label"].astype(int)

X = df["stemmed_title_abstract"]
y = df["Label"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=df["Label"]
)
```

Create predictive model

```
[26]: vectorizer = TfidfVectorizer(
    # stop_words = "english",
    ngram_range=(1, 2),
    # min_df = 2,
    # max_df = 0.95
)

X_train = vectorizer.fit_transform(X_train)
X_test = vectorizer.transform(X_test)

# Fit model
```

```
lr = LogisticRegression(max_iter = 1000)
model = lr.fit(X_train, y_train)

# Run predictive model
y_pred = model.predict(X_test)
```

Get the most frequently used words/phrases associated with included and excluded studies

```
[27]: feature_names = vectorizer.get_feature_names_out()
      coef = pd.Series(model.coef_[0], index = feature_names)
```

Included

```
[28]: coef.sort_values(ascending = False).head(50) # Included studies
```

```
[28]: tendon          0.909052
      qt              0.844233
      graft           0.825780
      patellar        0.802147
      patellar tendon 0.753751
      pt              0.639123
      ht              0.554911
      tendon group    0.538846
      autograft        0.490101
      tendon graft     0.421455
      patient patellar 0.413201
      li               0.387014
      quadricep tendon 0.362981
      tendon autograft 0.344327
      stg              0.343549
      hamstr           0.342632
      brake            0.342199
      precondit        0.341338
      semitendinosu    0.333099
      bptb             0.326383
      endobutton       0.326155
      bpt              0.321689
      degre sec        0.313665
      drain            0.310743
      grade            0.309181
      st               0.306576
      interfer fixat    0.299654
      sec              0.296666
      lb               0.279179
      kneel            0.274404
      patient          0.262074
      stiff            0.261870
```

year	0.258664
tendon hamstr	0.254971
type graft	0.254800
stg endobutton	0.251655
immedi post	0.250078
bone	0.249325
signific differ	0.247622
brt	0.244300
btt	0.244300
intern tibial	0.240648
health relat	0.238297
lk	0.236545
hip	0.233221
qt autograft	0.233153
compar healthi	0.226233
patient hamstr	0.211788
flexion strength	0.210541
menisc	0.207994
dtype: float64	

Excluded

```
[29]: coef.sort_values(ascending = True).head(50) # Excluded studies
```

```
[29]: group          -0.444784
      muscl          -0.319012
      allograft      -0.238328
      brace          -0.229215
      exercis        -0.229081
      screw          -0.224716
      bundl          -0.193535
      effect         -0.185275
      knee           -0.179935
      improv         -0.178951
      augment        -0.158130
      postop         -0.157176
      program        -0.156292
      control group  -0.155878
      group group    -0.154355
      limb           -0.151893
      week           -0.143617
      muscl strength -0.142364
      rehabilit      -0.134884
      stimul         -0.132152
      nerv           -0.131684
      tunnel         -0.128256
      train          -0.124169
```

therapi	-0.122777
singl bundl	-0.122567
repair	-0.122458
mm	-0.122424
femor	-0.122376
doubl	-0.122136
reduc	-0.117496
standard	-0.116746
metal	-0.116223
convent	-0.114975
evalu	-0.114934
earli	-0.113638
femor nerv	-0.113452
incis	-0.112895
sd	-0.111530
nerv block	-0.111412
let	-0.110684
widen	-0.110467
rate	-0.110413
ci	-0.109487
intervent	-0.109185
contralater	-0.108991
eccentr	-0.107021
activ	-0.105851
fnb	-0.105847
interfer screw	-0.104492
knee laxiti	-0.103550
dtype: float64	